

VISUALSENSE

Gesture Controlled Console

23-NTU-CS-1071 – Muqaddam Ali

23-NTU-CS-1060 – Muhammed Ahmed

23-NTU-CS-1074 – Talal Shariq

Submitted to: Dr. Nasir Mehmood
Embedded IoT Systems

Problem Statement and Objectives

Problem Statement

Traditional input methods like keyboards, mice, and touchscreens require physical contact, which can be limiting in specific IoT scenarios. There is a need for a touchless, and intuitive interface (maybe just for aesthetically pleasing or futuristic-looking purposes) that bridges the gap between physical hardware and digital control systems without relying on expensive dedicated sensors like Kinect or Leap Motion.

Basically just to feel futuristic and look cool. (IDK maybe like ironman or something)

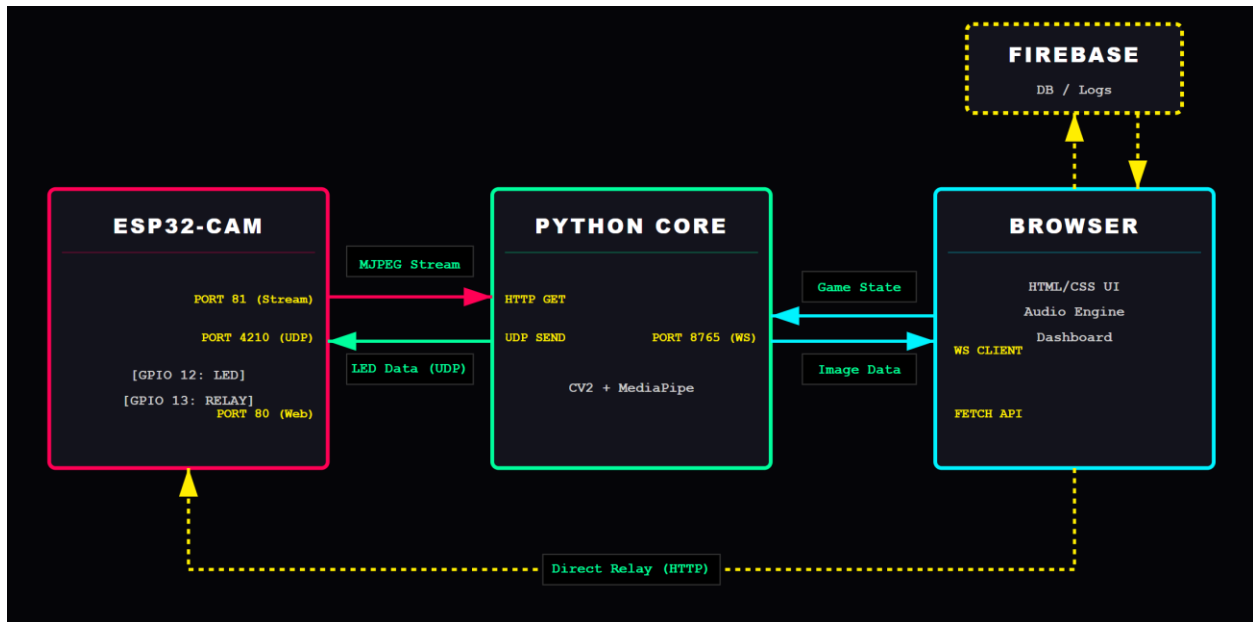
Objectives

The primary objectives of the VisualSense project are:

1. To design a wireless video streaming system using the ESP32-CAM module.
2. To implement real-time computer vision algorithms using Python and MediaPipe to track hand landmarks.
3. To create a bi-directional communication protocol where hand gestures control both a web-based game console and physical hardware components (LEDs and Relays).
4. To integrate a cloud database (Firebase) for persistent data storage, such as leaderboards and security logs.

System Architecture / Block Diagram

The system operates on a client-server model. The ESP32 acts as the video server and physical actuator. The PC acts as the processing unit, running the Python script to analyze the video feed. The Web Browser acts as the visual interface and game engine.



Hardware and Software Description

Hardware Components

- **ESP32-CAM (AI Thinker Model):** The core microcontroller that handles Wi-Fi connectivity and video capturing.
- **OV2640 Camera Module:** A 2MP camera sensor used to capture the video feed.
- **FTDI Programmer:** Used for uploading the firmware to the ESP32. We got the one made specifically for ESP32-CAM AI Thinker.
- **LED & Resistor:** Connected to GPIO 12 to demonstrate PWM dimming via gesture (Knob mode).
- **Relay Module:** Connected to GPIO 13 to demonstrate security access control (Unlock mode).
- **Power Supply:** A stable 5V source to power the ESP32-CAM. This is a pain in the neck.

Software Stack

- **Arduino IDE:** Used for writing the C++ firmware (Hand_Tracking.ino). It handles the web server, camera configuration, and UDP listening.
- **Python 3.x:** Runs the backend script (esp_script.py).
- **MediaPipe:** A Google framework used for efficient hand tracking and landmark detection.

- **OpenCV:** Used for image processing, flipping frames, and drawing the HUD overlay.
- **Web Technologies:** HTML5, CSS3 (Cyberpunk/Futuristic aesthetic), and JavaScript for the frontend game logic.
- **Firebase:** A Google cloud database used for storing high scores and door access logs.
- **WebSockets:** The protocol used for low-latency communication between the Python script and the Web Interface.

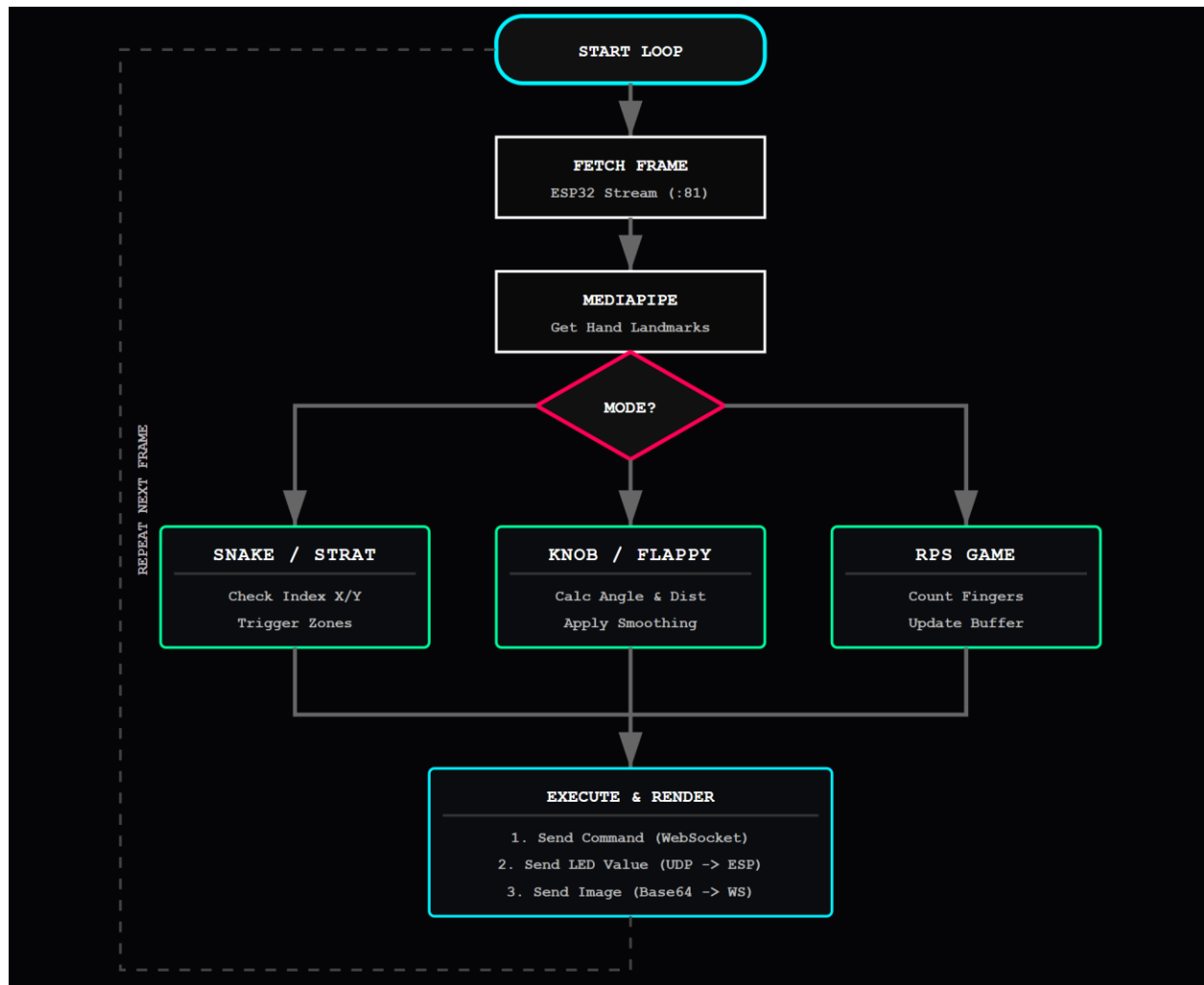
Methodology and Flowchart

Methodology

The project execution follows these steps:

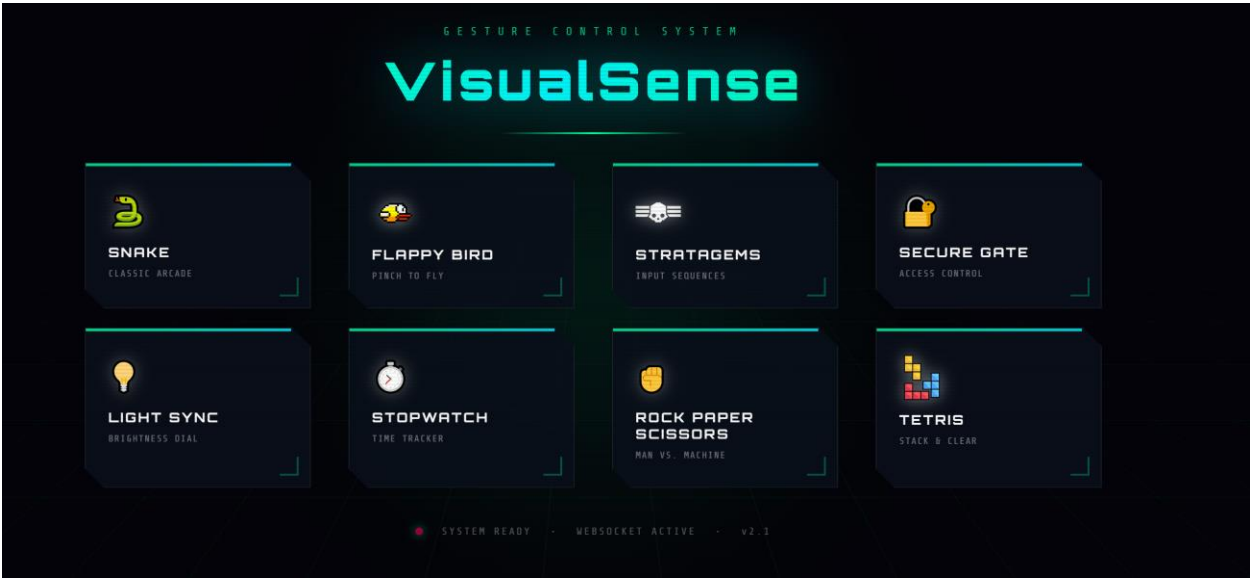
1. **Firmware Deployment:** The ESP32 is programmed to connect to WiFi and serve two HTTP ports. Port 80 serves the HTML interface, and Port 81 serves the raw MJPEG video stream.
2. **Stream Acquisition:** The Python script connects to the ESP32's stream URL. It decodes the raw bytes into video frames using OpenCV.
3. **Gesture Processing:** Each frame is passed to MediaPipe. If a hand is detected, 21 landmarks (coordinates) are extracted.
4. **Logic Calculation:**
 - **Snake/Tetris:** The system calculates the hand's position relative to center zones to determine Up, Down, Left, or Right commands.
 - **Knob:** The distance between the Thumb and Index finger determines the angle, which is smoothed mathematically to prevent jitter.
 - **RPS:** The number of extended fingers is counted to determine Rock, Paper, or Scissors.
5. **Command Transmission:**
 - Game commands are sent via WebSocket to the localhost browser to control the UI.
 - Hardware commands (like LED brightness) are sent via UDP directly to the ESP32 to reduce latency.

Flowchart

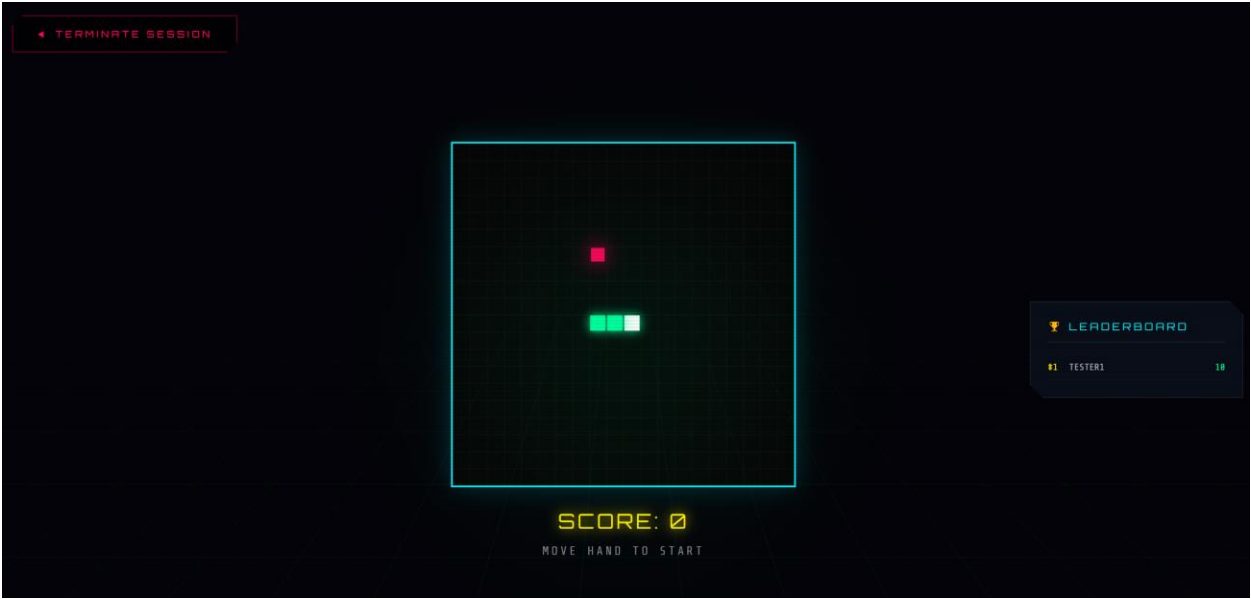


Screenshots of Output and Dashboards

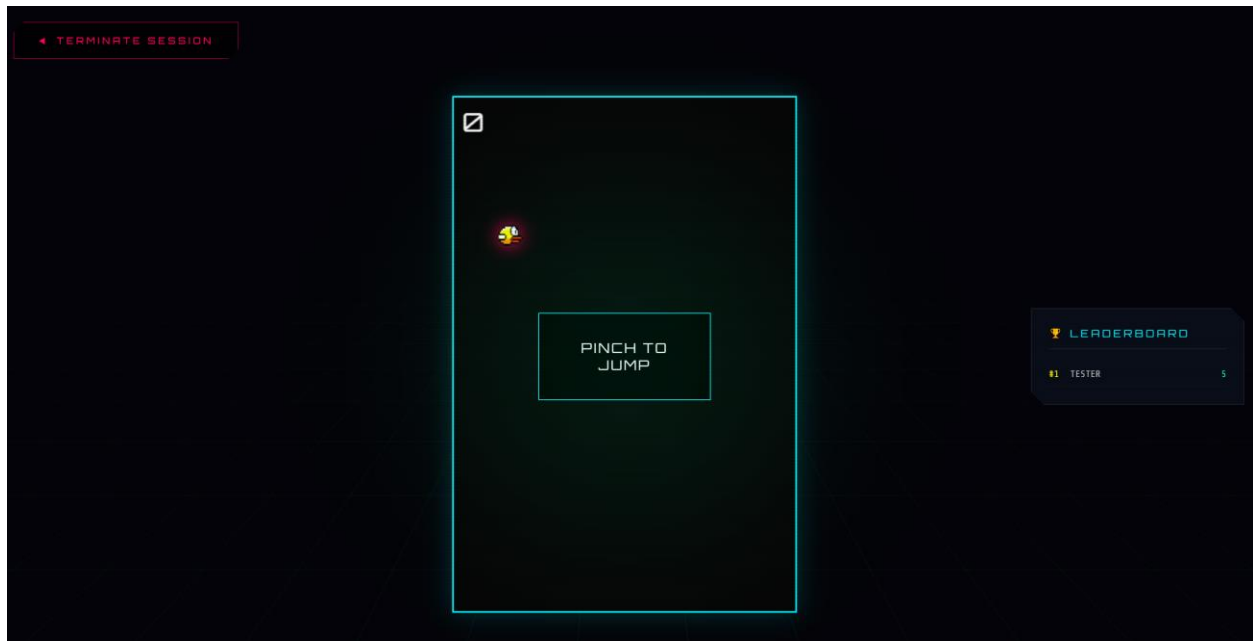
Main Dashboard



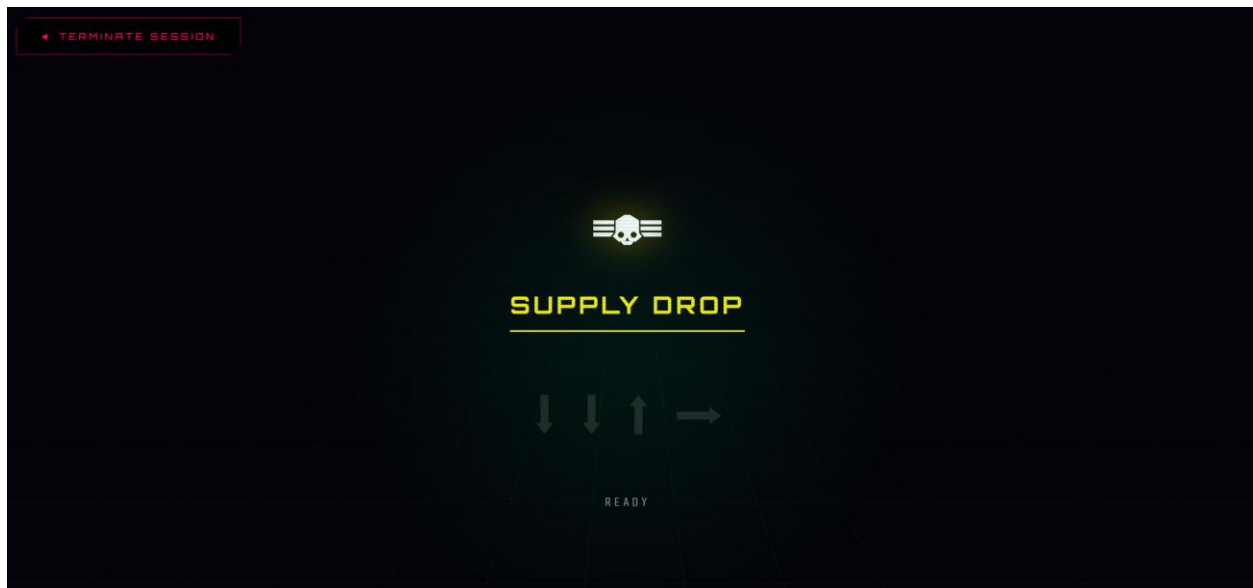
Snake



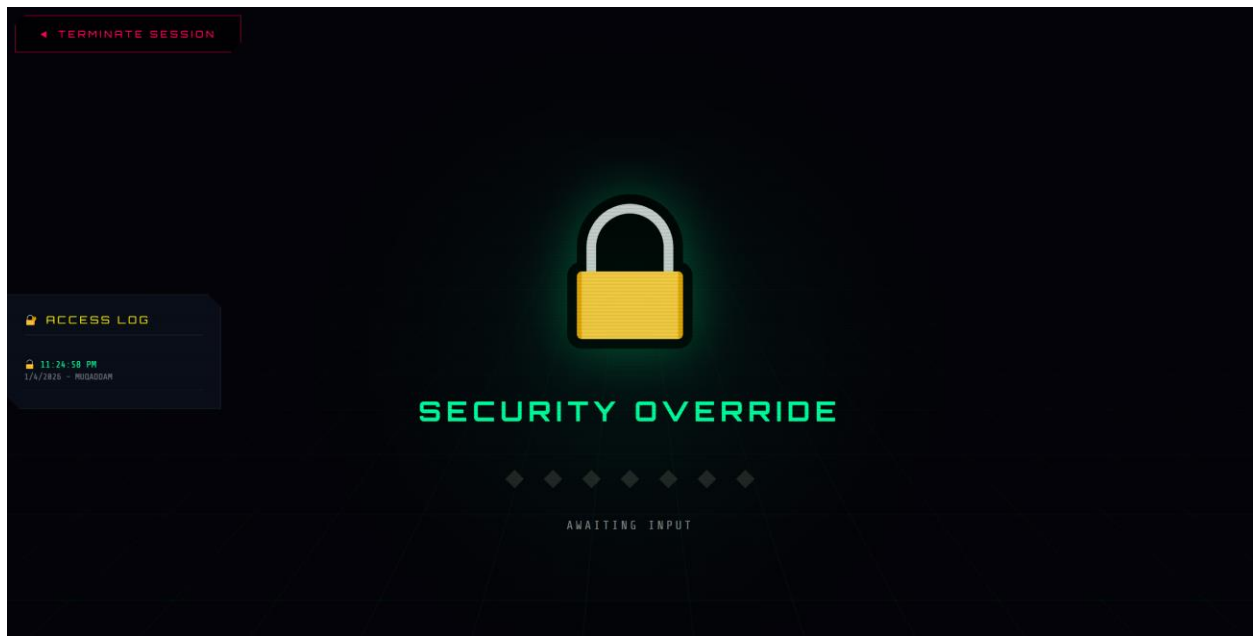
Flappy Bird



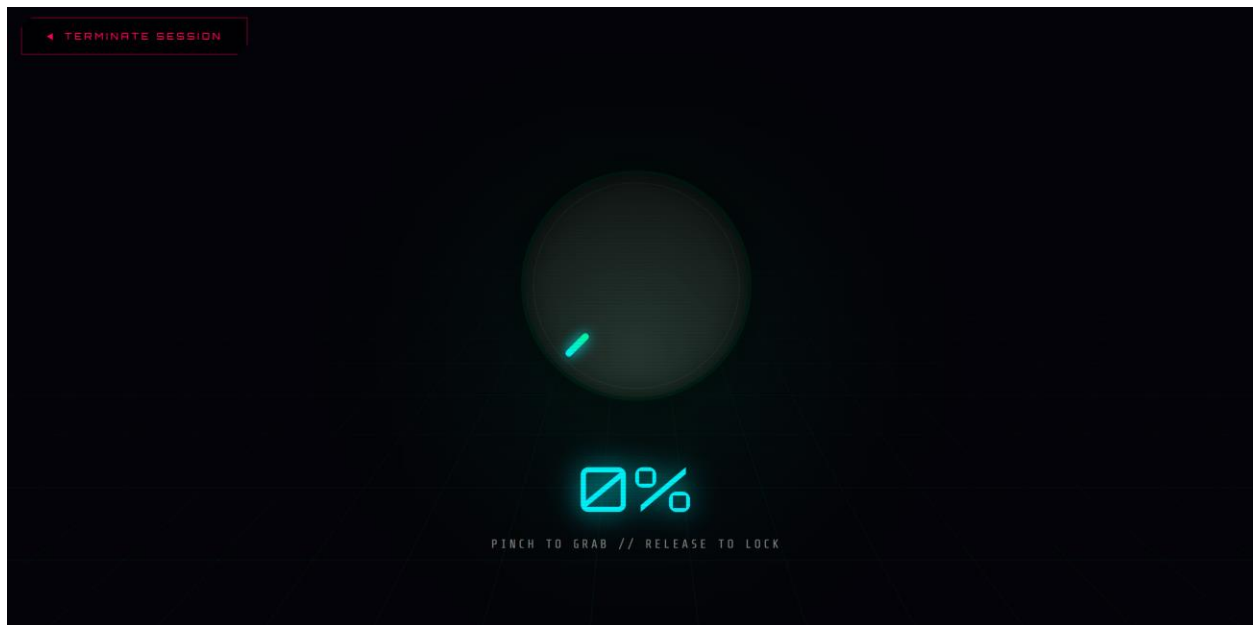
Stratagems



Door Lock/Security Override



Light Sync/Brightness Knob



Stopwatch

00:00:00

FIST = PAUSE | PALM = RUN

Rock Paper Scissors

◀ TERMINATE SESSION

MAN VS MACHINE

READY?



VS

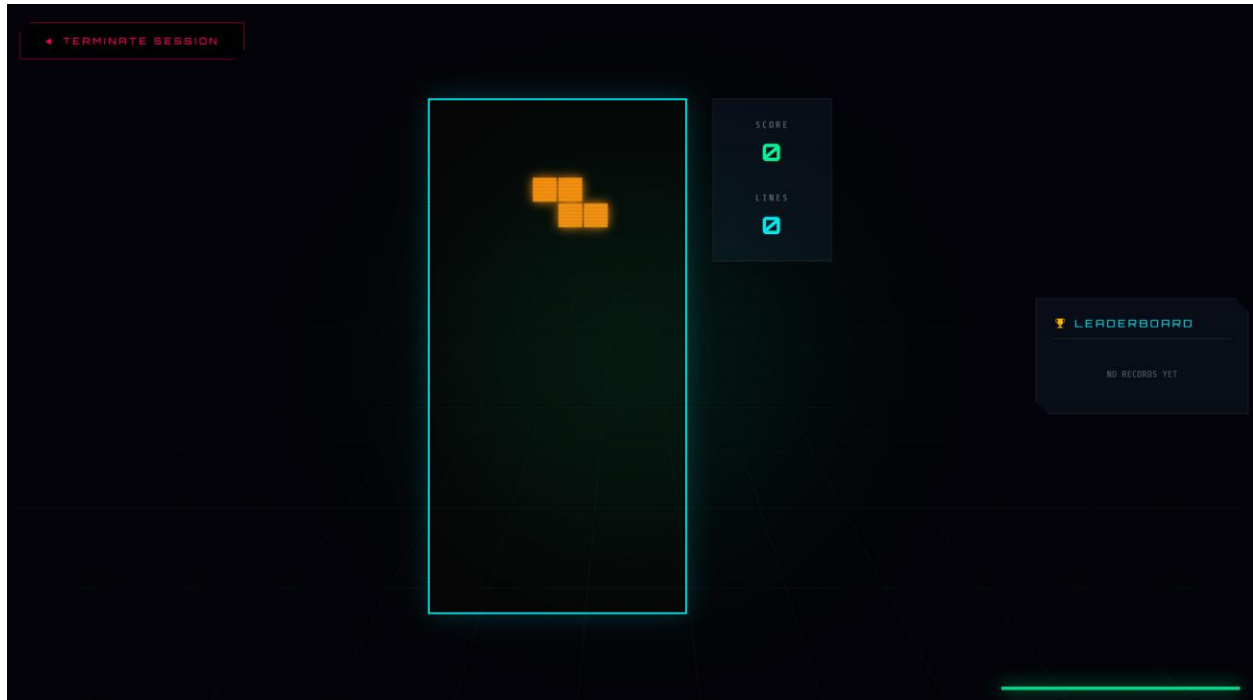


YOU

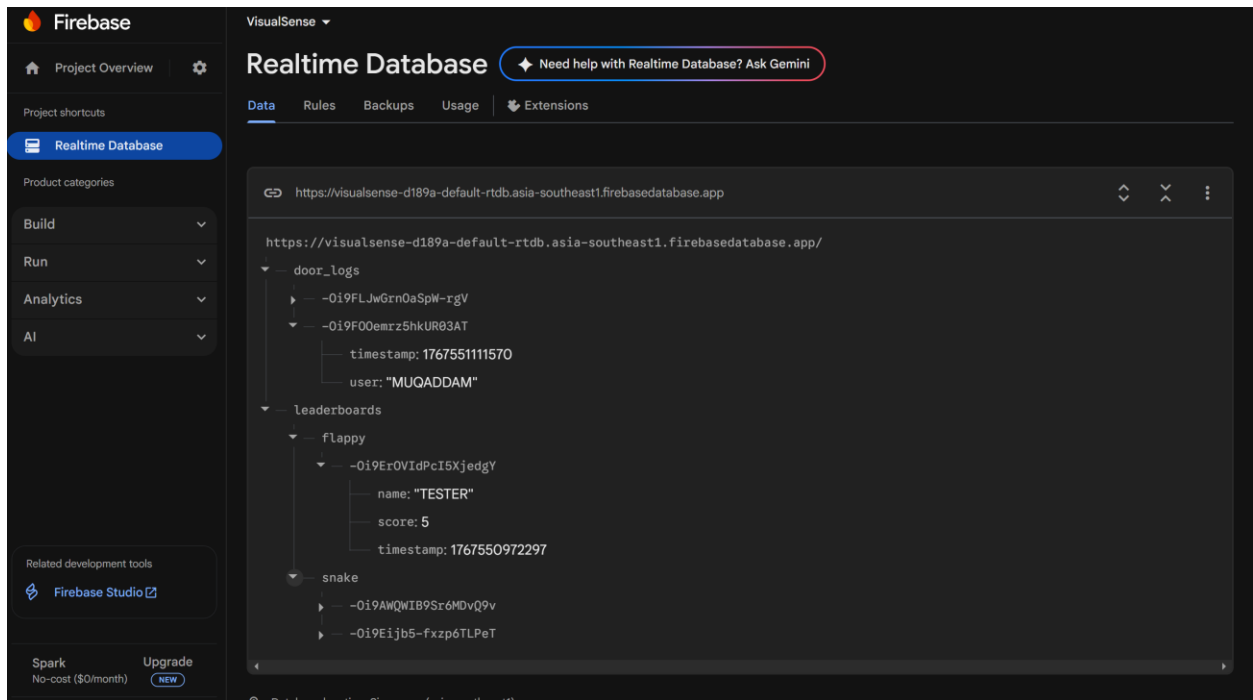
CPU

SHOW 'ROCK' (FIST) TO START

Tetris



Firebase/Cloud Database



Results, Conclusion, and Future Scope

Results

The VisualSense console successfully integrates eight distinct modes of operation. The system achieves a frame rate suitable for casual gaming (Snake and Tetris) and responsive hardware control. The "Knob" mode demonstrates smooth dimming of the physical LED using the implemented smoothing algorithm in Python. The Firebase integration successfully logs "Door Unlock" events with timestamps, fulfilling the security requirement.

Conclusion

This project demonstrates that standard ESP32 hardware, combined with modern computer vision libraries, can create a powerful gesture control system. By offloading the heavy image processing to a PC via Python, we bypassed the processing limitations of the microcontroller while maintaining wireless flexibility. The result is a robust, touchless console that serves both entertainment and home automation purposes.

Future Scope

1. **On-Chip Processing:** Porting the hand detection logic to the ESP32-S3/Raspberry Pi (which has AI acceleration) to remove the need for a PC backend.
2. **Voice Integration:** Adding voice commands to switch modes or control games alongside gestures.
3. **3D Navigation:** Expanding the gesture logic to support depth-based movements (Z-axis) for more complex interactions.
4. **Multi-Hand Support:** Enhancing the logic to support two-player games using simultaneous detection of both hands.